

Operating System Lab

Processes – Study Material

Essential Concepts for Lab Assignment 4

Scope. This material contains only the concepts required to solve Lab Assignment 4. The emphasis is on understanding what each system call does, why it is needed, and how the calls interact. It is not a replacement for the Processes chapter of the textbook.

Contents

1	Process: Program, Execution, and PCB	3
2	PID and PPID	3
3	Creating a Process with <code>fork()</code>	3
4	What is Copied by <code>fork()</code> ?	4
5	Reasoning About Process Trees	5
6	Concurrent Execution and Output Order	6
7	<code>waitpid()</code> and Process Synchronization	6
8	Process Termination and Exit Status	7
9	<code>exec()</code> and Process Image Replacement	8
10	What Changes and What Does Not Change with <code>exec()</code> ?	8
11	Why <code>exec()</code> Is Often Used After <code>fork()</code>	9
12	Multiple Children	9
13	Monitoring Multiple Children with <code>WNOHANG</code>	10
14	Pipes for Synchronization	12
15	Orphan Processes	13
16	Zombie Processes	14
17	Why <code>sleep()</code> Is Not Synchronization	14

18 Observing Processes in Linux	15
19 Common Mistakes to Avoid	15
20 Quick Reference	16
21 Final Summary	16

1 Process: Program, Execution, and PCB

A **program** is a passive collection of instructions and data stored on a storage device. A **process** is an executing instance of a program managed by the operating system.

A process has an execution context in addition to the program code. Important information includes:

- program counter;
- CPU register state;
- address-space information;
- scheduling information;
- process state;
- process identifier (PID); and
- information required for resource and I/O management.

The operating system maintains process-management information in a **Process Control Block (PCB)**. The exact implementation is operating-system dependent.

For this lab, the most important identifiers are the **PID** and **PPID**.

2 PID and PPID

A Linux process has a process identifier (PID). A process can obtain its own PID using:

```
1 #include <unistd.h>
2
3 pid_t pid = getpid();
```

A process can obtain the PID of its current parent using:

```
1 pid_t ppid = getppid();
```

Thus, if process P_1 is currently a child of process P_0 ,

$$PPID(P_1) = PID(P_0)$$

while that parent-child relationship exists.

The PPID is a relationship that can change during the lifetime of a process; it should not be treated as a permanent identifier of the process's original parent.

3 Creating a Process with fork()

In Linux/POSIX systems, `fork()` is used to create a new process.

```
1 #include <sys/types.h>
2 #include <unistd.h>
3
4 pid_t pid = fork();
```

A successful `fork()` creates one new child process. After the call, both parent and child continue execution from the instruction immediately following `fork()`.

The return value distinguishes the two processes:

Execution context	Return value
Parent	PID of the newly created child
Child	0
Failure	-1 (returned to the caller; no child is created)

A standard pattern is:

```

1 pid_t pid = fork();
2
3 if (pid < 0) {
4     perror("fork");
5     exit(EXIT_FAILURE);
6 }
7 else if (pid == 0) {
8     /* child */
9 }
10 else {
11     /* parent; pid is the child's PID */
12 }
```

Important: after a successful `fork()`, both processes execute the code after the call unless their control flow takes them elsewhere.

4 What is Copied by `fork()`?

The child initially receives a logically separate copy of the parent's address space. Therefore, variables have the same initial values in the two processes, but later modifications are not shared as ordinary memory.

For example:

```

1 int x = 10;
2
3 pid_t pid = fork();
4
5 if (pid == 0) {
6     x = 20;
7     printf("Child:  x = %d\n", x);
8 }
9 else {
10     printf("Parent: x = %d\n", x);
11 }
```

The child's modification of its copy of `x` does not directly modify the parent's copy.

Modern systems commonly implement this efficiently using **copy-on-write**: pages can initially be shared internally and copied when a process attempts to modify them. The programming model remains that the parent and child have logically separate address spaces.

5 Reasoning About Process Trees

A common mistake is to count `fork()` statements without asking which processes actually execute each statement.

For example:

```
1 fork();
2 fork();
```

If every process reaches both calls, the number of processes evolves as

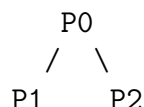
$$1 \rightarrow 2 \rightarrow 4.$$

The correct question for every `fork()` is:

Which existing processes reach this particular call?

To construct a required process hierarchy, explicitly separate the execution paths of the parent and child.

For example, to create:



the first child should not accidentally continue into code that creates the second child:

```
1 pid_t p1 = fork();
2
3 if (p1 < 0) {
4     perror("fork");
5     exit(EXIT_FAILURE);
6 }
7
8 if (p1 == 0) {
9     /* P1 work */
10    exit(EXIT_SUCCESS);
11 }
12
13 /* Only P0 reaches this point. */
14 pid_t p2 = fork();
15
16 if (p2 < 0) {
17     perror("fork");
18     exit(EXIT_FAILURE);
19 }
20
21 if (p2 == 0) {
22     /* P2 work */
23     exit(EXIT_SUCCESS);
24 }
25
26 /* P0 continues here. */
```

This is the key technique used in Task 1.

6 Concurrent Execution and Output Order

After `fork()`, parent and child are separate runnable processes. The scheduler decides when each process executes.

Therefore, this program:

```
1 pid_t pid = fork();  
2  
3 if (pid == 0)  
4     printf("Child\n");  
5 else  
6     printf("Parent\n");
```

may produce either:

Parent
Child

or:

Child
Parent

Both are valid.

On a single CPU core, execution may be interleaved; on a multicore system, different processes may execute simultaneously on different cores.

Therefore:

Creation order $\neq$ guaranteed execution order

Do not infer the process hierarchy from terminal output order. Use PID and PPID to determine parent–child relationships.

7 `waitpid()` and Process Synchronization

A parent may need to wait for a particular child to terminate. The `waitpid()` system call provides this functionality.

```
1 #include <sys/types.h>  
2 #include <sys/wait.h>  
3  
4 waitpid(child_pid, NULL, 0);
```

With `options = 0`, the call blocks until the specified child has terminated (or until an error occurs).

The three arguments are:

- `child_pid`: identifies the child to wait for;
- `status`: where the termination status can be stored, or `NULL` if it is not required;
- `options`: 0 gives normal blocking behavior.

For multiple children, the parent can wait for each required child:

```
1 waitpid(child1, NULL, 0);
2 waitpid(child2, NULL, 0);
3 waitpid(child3, NULL, 0);
```

Important distinction:

`waitpid()` is a synchronization mechanism. It does not tell the scheduler which process should receive the CPU first.

For example, if the parent calls `waitpid(child2, ...)` while `child1` has already terminated, the parent can still remain blocked until `child2` terminates.

8 Process Termination and Exit Status

A process can terminate explicitly using:

```
1 #include <stdlib.h>
2
3 exit(EXIT_SUCCESS);
```

or:

```
1 exit(EXIT_FAILURE);
```

Returning from `main()` also terminates the process. For example, `return 0;` from `main()` indicates successful termination.

The parent can collect a terminated child's status using `waitpid()`:

```
1 int status;
2
3 pid_t result = waitpid(child_pid, &status, 0);
4
5 if (result == -1) {
6     perror("waitpid");
7 }
8 else if (WIFEXITED(status)) {
9     printf("Exit status = %d\n", WEXITSTATUS(status));
10 }
```

Here:

- `WIFEXITED(status)` tests whether the child terminated normally;
- `WEXITSTATUS(status)` obtains the child's exit status when `WIFEXITED(status)` is true.

For example:

```
1 /* child */
2 exit(42);
```

and, after a successful `waitpid()`, the parent can obtain:

$$WEXITSTATUS(status) = 42.$$

Do not use the process exit status to return arbitrary computation results. On Linux, the value available through `WEXITSTATUS()` is limited to the exit-status range defined by the process-wait interface (typically 8 bits). For example, a value such as $10! = 3628800$ should not be returned directly using `exit(3628800)`. Print such a computation result separately and use the exit status to indicate success or failure.

9 `exec()` and Process Image Replacement

The `exec()` family is used to replace the current process image with another program. This is different from `fork()`:

<code>fork()</code>	creates a new process
<code>exec()</code>	replaces the current process image

A common pattern is:

`fork() → exec()`

The parent creates a child, and the child then executes another program. For example:

```
1 pid_t pid = fork();
2
3 if (pid == 0) {
4     execl("./task2_exec",
5           "task2_exec",
6           (char *)NULL);
7
8     perror("execl");
9     exit(EXIT_FAILURE);
10 }
```

If `execl()` succeeds, it does **not return** to the old program. The process begins executing the new program image.

If `execl()` returns, the call failed and the error should be handled.

10 What Changes and What Does Not Change with `exec()`?

A successful `exec()` replaces the process's program image. It does not create a new process.

Therefore, the PID remains associated with the same process:

$PID_{\text{before exec}} = PID_{\text{after exec}}$

The program code, data, stack, and other parts of the process image are replaced according to the new program.

The parent–child relationship also remains a relationship between the same processes; `exec()` itself does not create a new child.

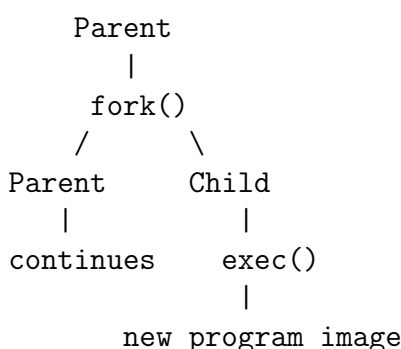
This distinction is central to Task 2:

`fork()` : new process `exec()` : new program image in the same process

11 Why `exec()` Is Often Used After `fork()`

A parent often needs to continue running one program while a child runs a different program.

The pattern is:



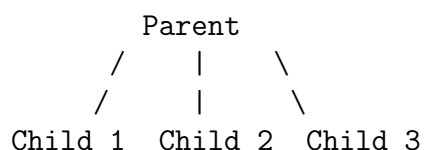
The child retains its PID after `exec()`, so the parent can still wait for that same child using its PID.

This is why Task 2 combines:

`fork() + exec() + waitpid()`

12 Multiple Children

Suppose a parent creates exactly three children:



The parent should retain the PIDs returned by the three successful `fork()` calls:

```

1 pid_t child_pids[3];
2
3 for (int i = 0; i < 3; ++i) {
4     pid_t pid = fork();
5
6     if (pid < 0) {
7         perror("fork");
8         exit(EXIT_FAILURE);
9     }
  
```

```

10
11     if (pid == 0) {
12         /* child-specific work */
13         exit(EXIT_SUCCESS);
14     }
15
16     child_pids[i] = pid;
17 }

```

The `exit()` in the child branch is important. Without it, a child may continue into the loop and create additional children.

The key point is that after a child is created, that child performs only its assigned work and then terminates. Therefore, only the original parent continues to the next iteration of the loop and creates the remaining children.

After creating the three children, the parent can wait for each stored PID.

This is the basic idea required in Task 4.

13 Monitoring Multiple Children with WNOHANG

For Task 4, the parent must monitor multiple child processes without blocking while waiting for a particular child.

The following form is used:

```

1 int status;
2
3 pid_t result = waitpid(-1, &status, WNOHANG);

```

The arguments have the following meaning:

- `-1`: wait for any child of the calling process;
- `&status`: store information about the child's termination;
- `WNOHANG`: return immediately if no child has terminated.

The return value must be interpreted carefully:

Return value	Meaning
<code>-1</code>	An error occurred.
<code>0</code>	No child has terminated yet.
<code>> 0</code>	PID of a child whose termination status has been collected.

For example:

```

1 pid_t result = waitpid(-1, &status, WNOHANG);
2
3 if (result == -1) {
4     perror("waitpid");
5 }
6 else if (result == 0) {
7     /* No child has terminated yet. */

```

```
8 }  
9 else {  
10     /* result is the PID of a terminated child. */  
11 }
```

The important distinction is:

`waitpid(child_pid, &status, 0)`

waits for a **specific child** and may block, whereas:

`waitpid(-1, &status, WNOHANG)`

checks whether **any child** has terminated and does not block when no child has terminated.

For Task 4, the parent can repeatedly perform this check until all three children have been collected.

Mapping a Returned PID to a Worker

Suppose the parent stores:

```
1 pid_t child_pids[3];
```

and `waitpid()` returns:

```
1 result = 12002;
```

The parent can compare `result` with the stored PIDs to determine which worker terminated.

Thus:

returned PID → corresponding child → completion order.

Completion Order

The order in which children are created does not determine the order in which they terminate.

For example:

Creation order:

Child 1 -> Child 2 -> Child 3

Possible completion order:

Child 2 -> Child 3 -> Child 1

The parent must therefore observe the actual completion order rather than assuming a fixed order.

Creation order $\neq$ Completion order

The completion order can depend on the amount of work performed by each worker, process scheduling, and the execution environment.

Monitoring Without Busy Waiting

If `waitpid()` returns 0, no child has terminated at that instant.

The parent may briefly suspend itself before checking again:

```
1 sleep(1);
```

The `sleep()` call only reduces unnecessary CPU usage. It does not determine whether a child has terminated.

The termination decision must come from `waitpid()`.

Collecting All Children

The parent must continue monitoring until every child has been collected.

For three children, the parent can maintain a count of completed children and stop only when:

$$\text{completed children} = 3.$$

This allows the parent to record the PID and exit status of each worker in the order in which the workers actually terminate.

14 Pipes for Synchronization

A pipe is a kernel-managed communication channel with a read end and a write end.

Create one using:

```
1 #include <unistd.h>
2
3 int fd[2];
4
5 if (pipe(fd) == -1) {
6     perror("pipe");
7     exit(EXIT_FAILURE);
8 }
```

Conceptually:

$$fd[0] = \text{read end}, \quad fd[1] = \text{write end}.$$

After `fork()`, both parent and child inherit the pipe descriptors. Each process should close the end it does not use.

A simple synchronization pattern is:

```
1 /* Child */
2 close(fd[0]);
3
4 /* perform observation */
5
6 char signal = 'X';
7 if (write(fd[1], &signal, 1) != 1) {
8     perror("write");
9     exit(EXIT_FAILURE);
```

```

10 }
11
12 close(fd[1]);

```

and:

```

1  /* Parent */
2  close(fd[1]);
3
4  char signal;
5  if (read(fd[0], &signal, 1) != 1) {
6      perror("read");
7      exit(EXIT_FAILURE);
8  }
9
10 close(fd[0]);
11
12 /* It is now safe for the parent to terminate. */

```

A blocking `read()` can therefore be used to make the parent wait until the child has sent the notification.

Important: the pipe is being used here for synchronization, not to transfer a computation result.

15 Orphan Processes

An **orphan process** is a process whose original parent terminates while the child is still running.

The important sequence is:

Parent terminates while Child continues running

Before the parent terminates:

$$PPID_{\text{child}} = PID_{\text{original parent}}.$$

After the parent terminates, the child is reparented by the operating system to an appropriate surviving process. On Linux this may be the system's init-like process or a configured subreaper; therefore, the new PPID should be **observed rather than assumed to be 1**.

The child's PID does not change merely because it becomes an orphan:

$$PID_{\text{before}} = PID_{\text{after}}$$

but its PPID can change.

For the lab, a pipe is used before the parent terminates so that the child can first record its original PID/PPID. A fixed `sleep()` is not a reliable substitute for this synchronization.

16 Zombie Processes

A **zombie process** is a child that has terminated, but whose parent has not yet collected its termination status.

The basic sequence is:

`fork()` → child terminates → zombie → `waitpid()` → reaped

A terminated child does not remain as a normally executing process. The kernel retains a small amount of process information, including information needed for the parent to collect the child's termination status.

While that information has not been collected, the process can appear in the process table with state:

Z

and may be displayed as `<defunct>`.

For example:

```
1 ps -o pid,ppid,state,stat,cmd -p <child_pid>
```

After the parent successfully calls `waitpid()`, the child is **reaped** and its process-table entry is removed. Therefore, after `waitpid()`, there is normally no process state to display for that PID.

A zombie is therefore fundamentally different from an orphan:

Orphan	Zombie
Child is still executing.	Child has already terminated.
Original parent has terminated.	Parent is still present but has not collected the child.
Child is reparented.	Termination information is retained temporarily.
Child has an active execution context.	Child is not executing.

17 Why `sleep()` Is Not Synchronization

The `sleep()` function suspends the calling process for approximately the specified duration:

```
1 sleep(2);
```

It can be useful for making process behavior easier to observe.

However:

`sleep()` ≠ synchronization

For example, if a child must record information before the parent terminates, writing:

```
1 sleep(2);
```

does not guarantee the required ordering.

The scheduler may run either process first, and machine load can affect timing.

Use a proper synchronization mechanism such as:

- `waitpid()` when waiting for child termination;
- a pipe when one process must explicitly notify another that an event has occurred.

This distinction is especially important in Task 4.

18 Observing Processes in Linux

The `ps` command can be used to inspect processes.

For example:

```
1 ps -f
```

A specific process can be inspected using:

```
1 ps -o pid,ppid,state,stat,cmd -p <PID>
```

A process hierarchy can also be inspected using:

```
1 pstree -p <PID>
```

Useful fields include:

- PID: process identifier;
- PPID: parent process identifier;
- STAT: process status information;
- CMD: command associated with the process.

For the zombie experiment, a state of Z indicates a zombie.

19 Common Mistakes to Avoid

1. Assuming only the child continues after `fork()`.

Both parent and child continue from the instruction after a successful `fork()`.

2. Counting `fork()` calls instead of execution paths.

Ask which processes reach each `fork()`.

3. Allowing a child to fall through into parent code.

Use an explicit child branch and terminate it after its intended work.

4. Assuming a fixed output order.

Scheduling makes the order of output from concurrent processes generally nondeterministic.

5. Confusing `fork()` and `exec()`.

`fork()` creates a process; successful `exec()` replaces the current process image.

6. Assuming `exec()` creates a new PID.

It does not create a new process, so the process keeps its PID.

7. Using `sleep()` as a synchronization mechanism.

Use `waitpid()` or a pipe when a real ordering guarantee is needed.

8. Using a large computation result as an exit status.

Use the exit status for success/failure and print large computation results separately.

9. Calling `waitpid()` for only one child when all children must be collected.

Keep the child PIDs and wait for each required child.

10. Assuming that an orphan's new PPID is always 1.

Observe the actual PPID on the Linux system. Subreapers can affect reparenting.

20 Quick Reference

Function	Purpose	Header
<code>fork()</code>	Create a new child process	<code>unistd.h</code>
<code>getpid()</code>	Obtain current PID	<code>unistd.h</code>
<code>getppid()</code>	Obtain current PPID	<code>unistd.h</code>
<code>waitpid()</code>	Wait for a particular child	<code>sys/wait.h</code>
<code>WIFEXITED()</code>	Test normal child termination	<code>sys/wait.h</code>
<code>WEXITSTATUS()</code>	Obtain normal exit status	<code>sys/wait.h</code>
<code>exec*()</code>	Replace current process image	<code>unistd.h</code>
<code>exit()</code>	Terminate the calling process	<code>stdlib.h</code>
<code>pipe()</code>	Create a pipe for communication/synchronization	<code>unistd.h</code>
<code>read()</code>	Read from a file descriptor	<code>unistd.h</code>
<code>write()</code>	Write to a file descriptor	<code>unistd.h</code>
<code>sleep()</code>	Suspend execution temporarily	<code>unistd.h</code>
<code>perror()</code>	Print an error message	<code>stdio.h</code>

21 Final Summary

The four tasks build on the following chain of ideas:

Process $\rightarrow$ `fork()` $\rightarrow$ Process Hierarchy $\rightarrow$ `waitpid()`

then:

`fork()` $\rightarrow$ `exec()` $\rightarrow$ same PID, new process image

then:

Parent–Child Lifetime → Pipe Synchronization → Orphan / Zombie

and finally:

Multiple Children → `exec()` → `waitpid(-1, &status, WNOHANG)` → Completion Order

The most important distinctions to remember are:

Concept	Meaning
<code>fork()</code>	Creates a new process.
<code>exec()</code>	Replaces the current process image.
<code>waitpid()</code>	Synchronizes with and collects a child.
<code>WNOHANG</code>	Prevents <code>waitpid()</code> from blocking when no child has terminated.
<code>pipe()</code>	Provides inter-process communication and synchronization.
<code>exit()</code>	Terminates the calling process.
Orphan	A child that continues running after its original parent terminates.
Zombie	A terminated child whose termination status has not yet been collected.